

PAXOS LABS · SECURITY REVIEW

Paxos Labs **Nucleus Boring Vault** **Review.**

A security review of Nucleus Boring Vault Review.

CLIENT

Paxos Labs

PROJECT

Nucleus Boring
Vault Review

AUDITORS

Guardian

AUDIT WINDOW

June 22 – June 24,
2026

FINAL REPORT

July 2, 2026

§ 00

Contents.

23 pages — overview, confidence ranking, scope, 11 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 11 findings	P. 08
§ 05	Disclaimer	P. 22
§ 06	About Guardian	P. 23

§ 01

Overview.

Guardian reviewed Nucleus Boring Vault Review across a single review round. The final report includes 2 Medium, 4 Low, 5 Informational findings.

AUDIT FIRM	METHODOLOGY
Guardian	Smart contract · Manual · Main Review
CLIENT	
Paxos Labs	
PROJECT	
Nucleus Boring Vault Review	
PREPARED BY	
Guardian	
AUDIT WINDOW	
June 22 – June 24, 2026	
FINAL REPORT	
July 2, 2026	

1.1 Confidence Ranking

5 / 5

Very High Confidence

Guardian assigns a Confidence Ranking of 5 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	0	0	0	0	0
Medium	2	0	1	0	1
Low	4	0	3	0	1
Informational	5	0	3	1	1
All severities	11	0	7	1	3

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence ASSIGNED	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Nucleus Boring Vault Review.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
M-01	Sub-dust order queues with zero amountDue	Validation	MEDIUM	ACKNOWLEDGED	p. 8
M-02	Refunded in-flight order can execute	Logical Error	MEDIUM	RESOLVED	p. 9
L-03	Version-skewed payload corrupts order	Logical Error	LOW	ACKNOWLEDGED	p. 11
L-04	Stale executePendingOrders decoder	Compatibility	LOW	RESOLVED	p. 13
L-05	Depeg controls don't gate queued fills	Validation	LOW	ACKNOWLEDGED	p. 14
L-06	Quote signer can redirect receiver	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 15
I-02	quoteSend fee wrong for unset gas limit	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 16
I-03	Stale receive-route validation docs	Documentation	INFORMATIONAL	PARTIALLY RESOLVED	p. 17
I-04	Disabled peer can backlog packets	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 18
I-06	Same-chain value natspec contradicts code	Documentation	INFORMATIONAL	RESOLVED	p. 19
I-09	One bad fill DoSes the whole batch	DoS	INFORMATIONAL	ACKNOWLEDGED	p. 20

MEDIUM **M-01**

ACKNOWLEDGED

Sub-dust order queues with zero amountDue

CATEGORY Validation
LOCATION TransitStation.sol:640-651
REVIEW Main Review

DESCRIPTION

`_pushOrder` derives the want owed as `amountDue = _toTokenDecimals(offerAmountNormalized18AfterFees, wantAsset.decimals())`. `_toTokenDecimals` floors toward zero, so when the offer asset has more decimals than the want and the collected net is small, `amountDue` truncates to zero (`TransitStation.sol:643`) and the order is still queued — no revert.

This contradicts the project's design documentation, which states that sub-dust orders revert. No such guard exists: there is no `ZeroAmountDue` error and no zero-check in `_pushOrder` or `_verifyAndCollect`. The only size guard, the fee-vs-offer partition (`TransitStation.sol:580`), ensures a net of just one offer-token unit — far below the want-truncation threshold.

The net was already pulled to `offerReceiver`, so the submitter paid while the queued order owes zero want. It is born "fully filled": `executePendingOrders` rejects any positive fill with `AmountExceedsDue` (due is 0), and a zero fill removes it delivering nothing. Worse, a dead order co-batched with healthy ones leaves the vault's approval partly unspent, tripping the per-batch `ResidualApproval` check and reverting the whole batch.

RECOMMENDATION

Enforce a minimum order size on the source chain, in `_verifyAndCollect`, so sub-dust orders are rejected before any funds move. This is the only safe fix for the bridged path: a destination-side revert there would strand a LayerZero message whose source funds were already collected. As a backstop for same-chain submissions, after deriving `amountDue` in `_pushOrder`, revert with a dedicated `ZeroAmountDue` error when it is zero — matching the behavior the design documentation already specifies.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

RESOLVED

Refunded in-flight order can execute

CATEGORY	Logical Error
LOCATION	src/transit/TransitStation.sol
REVIEW	Main Review

DESCRIPTION

Transit's admin removal/refund model relies on manual/off-chain refunds, but has no on-chain tombstone for cross-chain orders that were submitted on the source chain and have not yet been queued on the destination.

A cross-chain submit pulls the user's offer asset to offerReceiver, then sends `abi.encode(OrderTerms)` through LayerZero. The destination only records the order once `_lzReceive()` succeeds and calls `_pushOrder()`. If delivery is delayed or reverts before `_pushOrder()`, the order is in-flight but absent from `pendingOrderIds`.

In that state, `forceRemovePendingOrder(uuid)` cannot help because it requires the UUID to already be pending. If ops manually refunds the user while the LayerZero packet remains retryable, no Transit state marks the UUID as refunded. Later, after the delivery issue is fixed, the same authenticated packet can be retried. `_pushOrder()` only checks `pendingOrderIds.contains(terms.uuid)`, so it accepts the refunded UUID and queues it.

This is not limited to a one-off stuck packet. If LayerZero delivery or destination execution is degraded for a route, many source-chain submissions can accumulate as in-flight orders that are absent from destination `pendingOrderIds`. A user who observes the outage can submit multiple orders and later request refunds for all of them. If ops refunds from source-side records while those packets remain retryable, each packet can later be delivered and executed, multiplying the double-payout impact.

The executor can then fill the orders normally, causing users to keep the manual refund and also receive the want asset. The issue is that a refunded economic obligation is not made terminal on-chain.

LayerZero has packet-level recovery tools such as clear, nilify, and burn. The OApp or configured delegate can call these `EndpointV2` recovery functions, and burn makes a packet permanently unexecutable when its preconditions are met. However, Transit does not enforce use of any packet-level recovery before refunds and does not maintain an application-level refunded/cancelled UUID set. Safety therefore depends entirely on off-chain runbook discipline.

M-02 Refunded in-flight order can execute

CONTINUED

RECOMMENDATION

Add terminal state for refunded/cancelled UUIDs, checked in `_pushOrder()` before adding to `pendingOrderIds`. Add an admin function to mark in-flight UUIDs as cancelled/refunded before any off-chain refund. If `forceRemovePendingOrder()` is part of a refund flow, it should tombstone the UUID too. Operationally, require manual refunds to either tombstone the UUID in Transit or prove the exact LayerZero packet has been terminally cleared, burned, skipped, or otherwise made unexecutable first. nilify can be used as a temporary quarantine, but refunds should not rely on it as the final terminal state unless the packet cannot later be re-verified and executed.

RESOLUTION

Status: Resolved.

LOW **L-03**

ACKNOWLEDGED

Version-skewed payload corrupts order

CATEGORY	Logical Error
LOCATION	src/transit/TransitStation.sol
REVIEW	Main Review

— DESCRIPTION

Transit sends cross-chain orders as raw `abi.encode(OrderTerms)` bytes. The receiver authenticates only the local LayerZero endpoint and configured peer, then decodes the bytes directly as the current `OrderTerms` type.

There is no Transit-level message type, schema version, type hash, or exact payload-length check. The LayerZero/OApp version exposed by `oAppVersion()` is not a Transit payload schema version and is not checked before decoding application message bytes.

This is unsafe during migrations of normal non-upgradeable deployments. Transit versions are deployed as new contracts, and `OrderTerms` is the cross-chain wire format. If a source and destination station are ever configured across different `OrderTerms` schemas, Solidity ABI decoding will not fail closed by itself.

Solidity ABI encoding is not self-describing and requires the decoder to know the expected schema. For example, a payload encoded as `(bytes32,address,address,uint32,address,uint256)` is accepted when decoded as the current `(bytes32,address,address,address,uint256)`. The fields shift silently: current `offerAsset` becomes the old `sourceEID` word interpreted as an address, and current `offerAmountNormalized18AfterFees` becomes the old `offerAsset` address interpreted as a `uint256`. The trailing amount word is ignored.

This can happen during staggered redeloys or peer rotation if a new receiver is configured to trust an old-schema source, or if old-schema messages are not drained before changing the trusted source/destination pairing. Instead of failing closed, the destination can queue a corrupted order that requires manual cleanup. If executor automation fills from pending order data without reconciling against source events or backend records, the corrupted amount may also cause an incorrect payout.

— RECOMMENDATION

L-03 Version-skewed payload corrupts order

CONTINUED

Reject old-schema payloads during receive and make migrations drain-first. For the current static OrderTerms layout, require `payload.length == 160` before decoding so old six-word payloads fail closed instead of shifting fields. During redeploys or peer rotations, drain, retry, or clear in-flight packets from the old schema before configuring the new station to trust that peer. If Transit later expects multiple live message formats, add an explicit schema version at that point.

— RESOLUTION

Status: Acknowledged.

LOW **L-04**

RESOLVED

Stale executePendingOrders decoder

CATEGORY	Compatibility
LOCATION	NucleusDecoderAndSanitizer.sol:194-202
REVIEW	Main Review

— DESCRIPTION

`NucleusDecoderAndSanitizer` declares `executePendingOrders(bytes32[],uint256[])` (`NucleusDecoderAndSanitizer.sol:194`), selector `0xb4fb0493`, but `TransitStation.executePendingOrders` now takes `FillBatch[]`, selector `0xd9fe0258`. The decoder was never realigned when the fill input changed to per-asset batches.

Production fulfilment drives the vault through `ManagerWithMerkleVerification`, which staticcalls the decoder keyed by `bytes4(targetData)`. A `FillBatch[]` call matches no decoder function, falls through to the `BaseDecoderAndSanitizer` fallback, and reverts `FunctionNotImplemented`, reverting the whole manage call. The `submitOrder` decoder is aligned, so manager-driven deposits work while every manager-driven fill is bricked.

This is the live fulfilment path, not a hypothetical: fills currently run through the manager against a wired root. It is masked only because the deployed stations are pre-redesign bytecode whose `executePendingOrders` is still `0xb4fb0493`, matching the stale decoder. Deploying the audited `FillBatch[]` contract behind the unchanged decoder halts all merkle-gated fulfilment, stranding queued orders whose deposit was already collected.

— RECOMMENDATION

Realign the decoder to the current signature — declare `executePendingOrders((address,bytes32[],uint256[]))` in `NucleusDecoderAndSanitizer` so its selector matches `FillBatch[]` and the merkle leaf sanitizes each batch's `wantAsset`. Add a selector-parity check to the build so a future `TransitStation` signature change cannot silently desync the decoder again.

— RESOLUTION

Status: Resolved.

LOW **L-05**

ACKNOWLEDGED

Depeg controls don't gate queued fills

CATEGORY Validation
LOCATION TransitStation.sol:326
REVIEW Main Review

DESCRIPTION

Transit settles every approved route at a fixed 1:1 with no on-chain peg, oracle, or freshness check (TransitStation.sol:326). The 1:1 assumption is an accepted owner trust — only like-valued pegs are approved. The issue is not the depeg risk itself, but that the controls documented to contain a depeg fail for in-flight and queued cross-chain orders.

Route de-approval via `setRouteApprovals` is enforced only at submit on the source (TransitStation.sol:569); neither `_lzReceive` nor `executePendingOrders` re-checks the route, so de-approving a depegged route stops new submissions but leaves already-queued orders fully fillable. Halting the backend signer only blocks new quotes — quotes already signed are bearer instruments and, once queued, never expire (the `deadline` is an off-chain policy with no on-chain cap).

The only remaining on-chain control is `pause()`, which is global: stopping one depegged route stops every route and token, and there is no per-route execute freeze. Until an operator pauses globally, an arbitrageur's queued order on the depegged route fills 1:1 and `wantAssetSource` absorbs the full depeg spread on each.

RECOMMENDATION

Add an execution-side guard so a depegged route cannot be filled: re-check `approvedRoutes` inside `executePendingOrders`, and/or gate fills on a per-route freshness window or a peg sanity bound. Make route de-approval effective against already-queued orders — for example a per-route execute-side freeze — so incident response does not depend on a global `pause` that also halts healthy routes.

Cap the `Quote` `deadline` on-chain and/or add per-order expiry so pre-signed quotes cannot outlive a depeg response. Document that cross-chain depeg response requires pausing every peer station, since source-side de-approval does not protect orders already in flight.

RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

Quote signer can redirect receiver

CATEGORY Unexpected Behavior
LOCATION TransitStation.sol
REVIEW Main Review

— DESCRIPTION

Transit's docs model quoteSigner as a hot lower-trust key and state that a compromised signer is bounded by derived amountDue, approved routes, and the 10.5% combined fee cap. However, the signed Quote also contains receiver, and Transit only checks that it is nonzero.

submitOrder pulls the offer asset from msg.sender, then stores quote.receiver in OrderTerms. Later, executePendingOrders pays the want asset from wantAssetSource to that stored receiver. A compromised signer/backend can therefore return a valid quote with an attacker-controlled receiver. If a user submits it, their offer asset is collected to offerReceiver, while the eventual fill pays the want asset to the attacker. This is not bounded by MAX_PROTOCOL_FEE_BPS or MAX_INTEGRATOR_FEE_BPS, because the loss is not encoded as a fee. The entire payout address is controlled by the same backend signature. The bearer-quote design makes stolen quotes self-griefing only when the signed receiver is honest; it does not protect users from a malicious signed receiver unless the frontend independently verifies quote.receiver against the user's intended recipient.

— RECOMMENDATION

Bind the payout receiver to an independently trusted user intent. For direct user submits, require quote.receiver == msg.sender. If relayers or gas sponsors are required, add an explicit user-signed intent or trusted-forwarder scheme that binds the intended receiver separately from the backend quote. At minimum, update the compromised-quoteSigner risk model to state that signer compromise can redirect the full payout, not only skim capped fees.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

quoteSend fee wrong for unset gas limit

CATEGORY Documentation
LOCATION TransitStation.sol:465-476
REVIEW Main Review

DESCRIPTION

`quoteSend` builds the LayerZero fee estimate from `messageGasLimit[destEID]` with no zero-check and returns it (`TransitStation.sol:470`); its `@dev` states the estimate "matches the real send cost." But `_sendOrder` reverts `GasLimitNotSet` when the limit is zero. For an unconfigured destination EID, `quoteSend` silently returns a fee computed for a zero-gas option while the real cross-chain `submitOrder` reverts, so the documented equivalence is false exactly when the destination is not yet configured.

No value is at risk — the real send reverts safely — but an integrator relying on the quote for an unconfigured EID receives a misleading non-reverting fee.

RECOMMENDATION

Make `quoteSend` revert `GasLimitNotSet` when `messageGasLimit[destEID] == 0`, mirroring `_sendOrder`, so the preview and the real send agree.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03

PARTIALLY RESOLVED

Stale receive-route validation docs

CATEGORY	Documentation
LOCATION	Docs
REVIEW	Main Review

— DESCRIPTION

Some Transit documentation still states that the destination station's `_lzReceive` validates the route before recording a bridged order. Current code does not do this: `_lzReceive` decodes `OrderTerms` and calls `_pushOrder`, while route approval is enforced only during source-side submission. Newer sections of the same documentation also say the receive-side route check was intentionally removed, so the docs are internally inconsistent.

— RECOMMENDATION

Update the stale documentation to consistently state that route approval is checked only on the source station.

— RESOLUTION

Status: Partially resolved.

INFORMATIONAL **I-04**

ACKNOWLEDGED

Disabled peer can backlog packets

CATEGORY Warning
LOCATION OAppAuthReceiver.sol
REVIEW Main Review

DESCRIPTION

Transit's committed development notes describe `setPeer(eid, 0)` as a per-chain kill-switch after `PeerChain` was removed. In practice, this only blocks app-level delivery into Transit. It does not prevent LayerZero V2 from verifying and storing packets for an already-initialized path.

`OAppAuthReceiver.allowInitializePath()` returns true only when `peers[srcEid] == origin.sender`, and `IzReceive()` reverts when the configured peer is zero. However, after a path has delivered once, LayerZero endpoint verification no longer depends on `allowInitializePath()`: `_initializable()` returns true when `lazyInboundNonce > 0`. Therefore, packets from the old peer can still be verified and stored in `inboundPayloadHash` while `peers[srcEid] == 0`.

This is not an authentication bypass and does not directly move funds. The disabled peer still prevents `IzReceive()` from queuing the order while the peer is zero. The operational risk is that if operators later restore the same peer without clearing the endpoint backlog, packets verified during the disabled window can be delivered and queue orders.

Impact: Operators may misunderstand `setPeer(eid, 0)` as a full per-chain quarantine. For initialized LayerZero paths, it is only an application-level receive block; retryable verified packets can still accumulate at the endpoint and become executable after the peer is restored.

RECOMMENDATION

Document this caveat and include a restore runbook. Before re-enabling the same peer, inspect endpoint backlog for `(receiver, srcEid, oldPeer)` and clear, nilify, skip, or burn any packets that should not execute. For stronger protection, add an application-level peer epoch or high-water mark and reject packets from stale epochs/nonces even if LayerZero later delivers them.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-06

RESOLVED

Same-chain value natspec contradicts code

CATEGORY Documentation
LOCATION TransitStation.sol:237
REVIEW Main Review

DESCRIPTION

The `submitOrder` NatSpec states that the function is `payable` to fund the LayerZero native fee and that this value is "unused or refundable for same-chain" (`TransitStation.sol:237`). The code neither ignores nor refunds value on a same-chain order — it reverts. `_submitOrder` rejects any nonzero `msg.value` on the same-chain branch (`destEID == thisChainEID`) with `SameChainOrdersRequireNoValue` (`TransitStation.sol:547`).

A caller relying on the documented "refundable" behavior has the transaction revert instead of succeeding with a refund. For example, an integrator that passes a single fee estimate uniformly to both same-chain and cross-chain submits hits this revert. No funds are at risk; the defect is the mismatch between the documented contract and the actual behavior.

RECOMMENDATION

Correct the `submitOrder` NatSpec to state that a same-chain order must be submitted with `msg.value == 0`, and that any nonzero value reverts with `SameChainOrdersRequireNoValue`. If a uniform calling convention across both paths is preferred instead, ignore rather than reject `msg.value` on the same-chain branch; reverting is the safer default, so updating the documentation is the recommended fix.

RESOLUTION

Status: Resolved.

INFORMATIONAL I-09

ACKNOWLEDGED

One bad fill DoSes the whole batch

CATEGORY	DoS
LOCATION	TransitStation.sol:326
REVIEW	Main Review

DESCRIPTION

`executePendingOrders` (`TransitStation.sol:326`) settles a batch in a single all-or-nothing loop, calling `safeTransferFrom` per order with no per-fill `try/catch`. If any one fill reverts, the whole batch reverts — blocking every unrelated, healthy order grouped into it.

Two reachable triggers make a fill revert. The production want asset is a USDG BoringVault whose `transferFrom` runs a compliance freeze hook that reverts for a frozen recipient. A freeze of one queued order's receiver then reverts the whole batch — something the backend cannot predict.

Also, an order with `receiver == address(0)` transfers to the zero address, which most ERC20s reject. `_verifyAndCollect` guards `receiver != 0` on the source, but `_pushOrder` does not re-check it, so a forged bridged order (past LayerZero peer auth) can plant a zero-receiver order that bricks its batch.

The atomic revert is intentional — all-or-nothing avoids silent per-fill failures — and funds are not lost: the backend rebuilds the batch without the offending order, or an admin calls `forceRemovePendingOrder`. The residual gap is a transient liveness block of healthy orders co-batched with the bad one, not permanent loss.

RECOMMENDATION

Keep the all-or-nothing revert — it is a deliberate choice to avoid silent, hard-to-detect fill failures. Fix the avoidable triggers instead of swallowing reverts.

Before assembling a batch, have the backend screen each order's `receiver` against the want asset's compliance freeze list and exclude any frozen recipient. Keep batches small to bound the blast radius of an unexpected revert. Add a `receiver != address(0)` re-check in `_pushOrder` so a forged zero-receiver order can never queue. When a batch still reverts, the executor rebuilds it without the offending order or calls `forceRemovePendingOrder` to clear it.

I-09 One bad fill DoSes the whole batch

CONTINUED

— **RESOLUTION**

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.